Register Transfer & Micro-operations; **A**rithmetic **L**ogic **U**nit

uOttawa
L'Université canadienne
Canada's university

Université d'Ottawa | University of Ottawa



www.uOttawa.ca

Outline

- Register Transfer Language
- Register Transfer
- Bus & memory transfer
- Arithmetic Micro-operations
- Logic and Shift Micro-operations
- ALU Hardware Implementation

Register Transfer Language

- The internal hardware organization of a digital system, such as a digital computer for instance, is best defined by specifying:
  1. The set of registers it contains and their functions.
  2. The set of micro-operations that can be performed on the binary information stored in each register.
  3. The control that initiates the sequence of such micro-operations.
- An operation executed on data stored in registers within **one** clock cycle is called **micro-operations**.
- Shift, count, clear, and load, are examples of micro-operations.
- The symbolic notation used to describe the micro-operation transfers among registers is called a **register transfer language**.

Registers

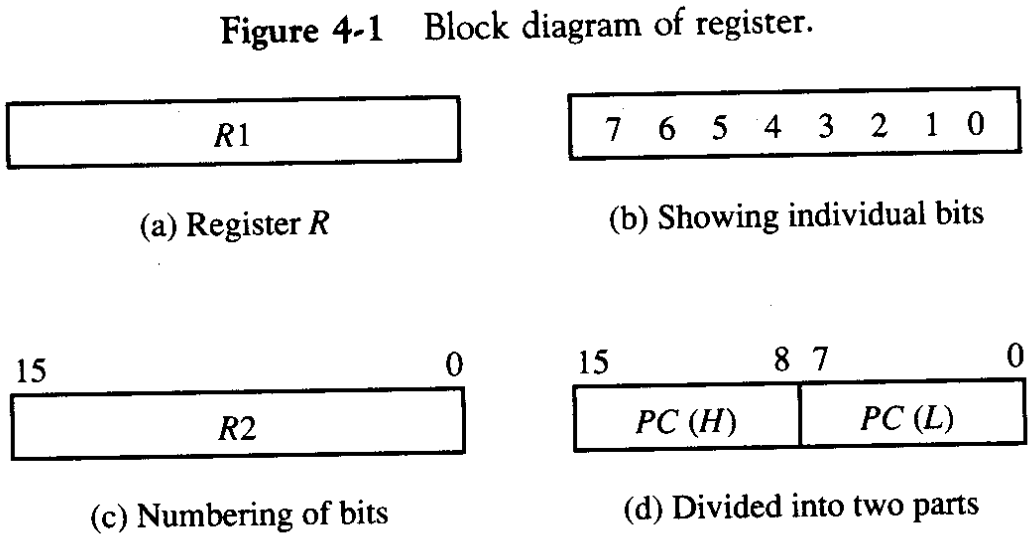


Figure 4-1 Block diagram of register.

(a) Register *R* (b) Showing individual bits

(c) Numbering of bits (d) Divided into two parts

- Computer registers are designated by capital letters possibly followed by numerals.
  - R1: general purpose data register.
  - MAR (Memory Address Register): a register to hold the address of the word to be fetched from memory.
  - PC: program counter register.
  - IR: instruction register.
- In the figure above, PC(L), or PC(0 -7), refers to the low-order byte, whereas PC(H), or
- PC(8 -15), refers to the high-order byte.

Register Transfer

- The statement R2 <- R1 denotes a transfer of the content of register R1 into register R2.
- By definition, the content of the source register, R1 does not change after the transfer.
- The statement R2 <- R1, R1 <- R2 denotes that the two transfer operations are to be executed at the same time (by the same clock impulse). In this particular case, it leads to the exchange of the contents of R1 and R2.

Basic symbols for register transfers

Symbol	Description	Examples
Letters (and numerals)	Denotes a register	MAR; R2
Parentheses ( )	Denotes a part of a register	R2(0-7), R2(L)
Left arrow	Denotes transfer of information	R2 <- R1
Comma ,	Separates two micro-operations	R2<-R1 , R5 <-MAR

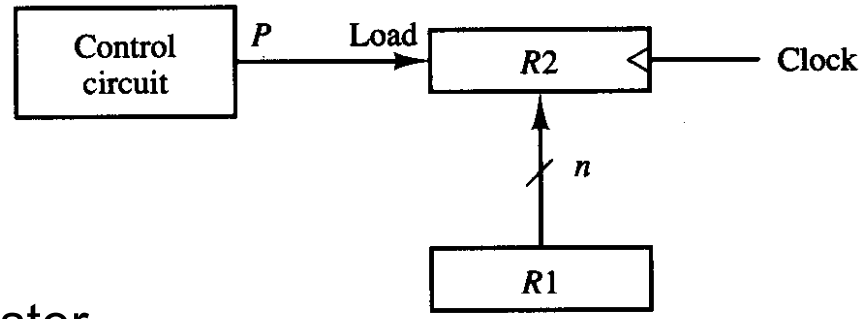
Control Function

- A control function is a Boolean variable. It is included in a register transfer statement as follows:

  P : R2 <- R1

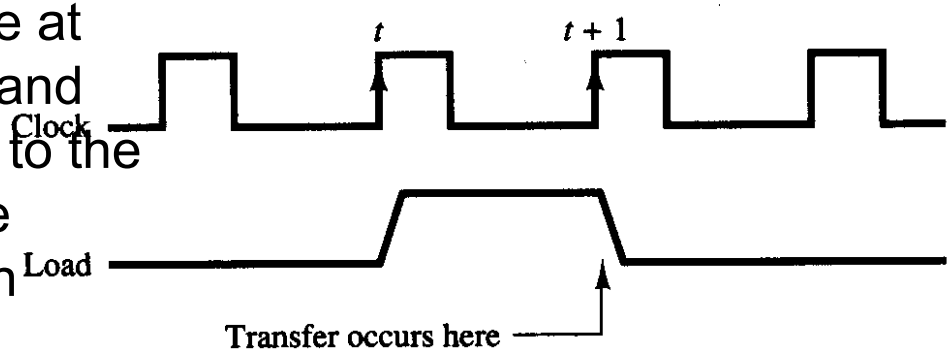
- The above statement means:

  if (P = 1) then (R2 <- R1)



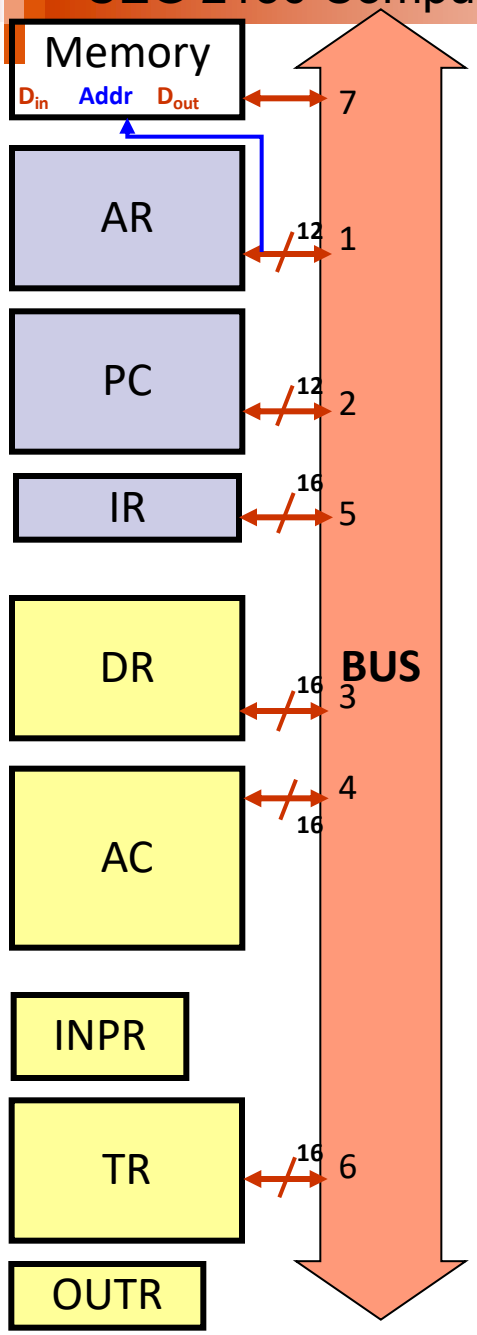
(a) Block diagram

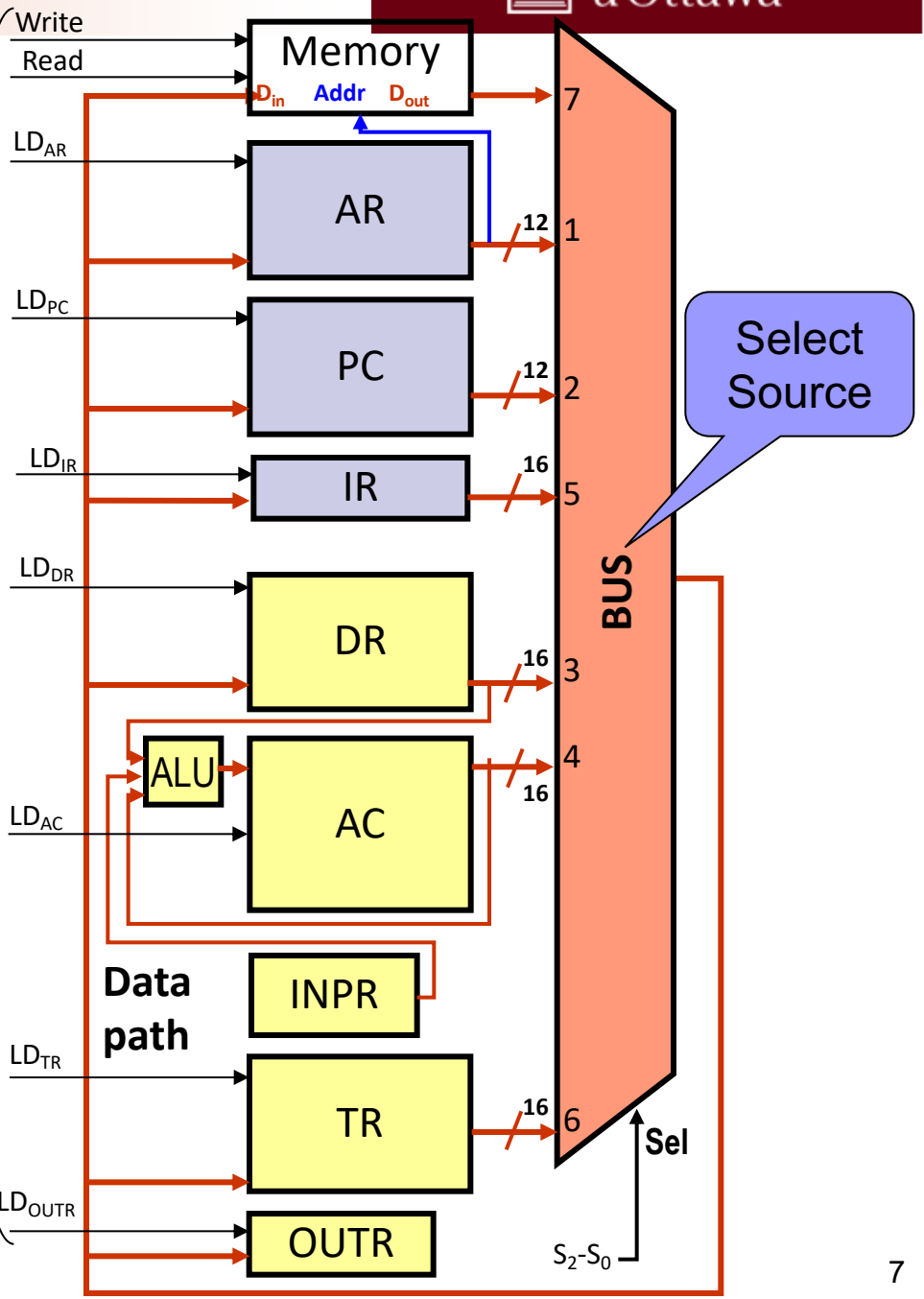
- *n* denotes the number of bits in a register.
- P is activated by the rising edge of a clock pulse at time t.
- The next transition of the clock pulse at time t + 1 finds the load input active and the data input lines of R2 connected to the data output lines R1. At this time, the contents of R1 gets loaded into R2 in parallel (parallel loading).



Bus & memory transfers

TR ← AC
(destination ← source)





Info Transfer

- The transfer of information from a bus into one of many destination registers can be accomplished by connecting the bus lines to the inputs of all destination registers and activating the load control bit of the selected destination register.
- The transfer of information between two registers through the bus can be symbolized explicitly by

$$BUS \leftarrow C\,, \quad R1 \leftarrow BUS$$

- or implicitly by

$$R \leftarrow C$$

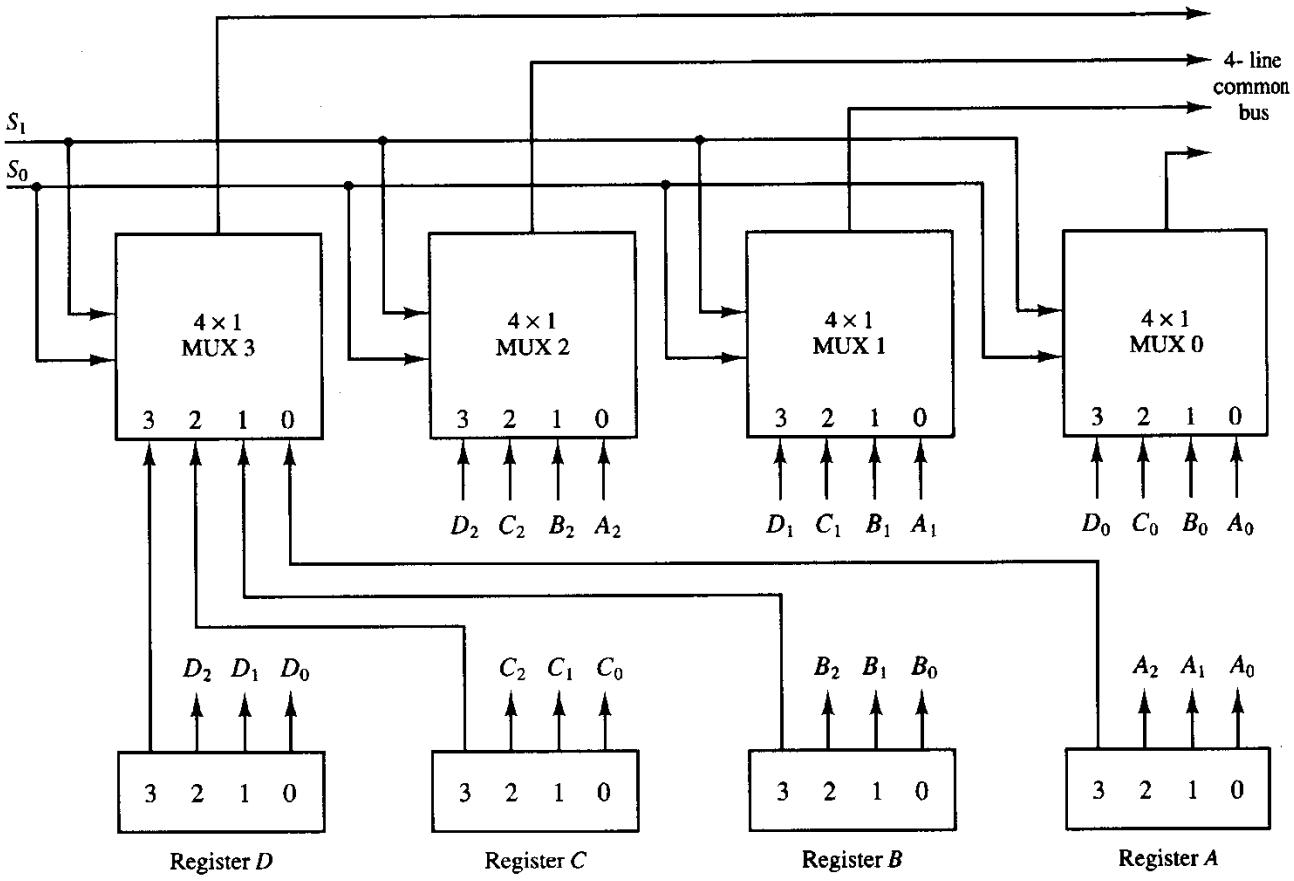
Common Bus With Multiplexers

A **bus** is employed to select the **source** of information and transfer it to a **destination.**
An *n*-line bus multiplexing 2^k registers is composed of:

- 2^k registers with *n* bits each (since it is an *n*-line bus)
- *n* 2^k × 1-multiplexers: each multiplexer has 2^k data bits, *k* selection bits, and 1 output bit.

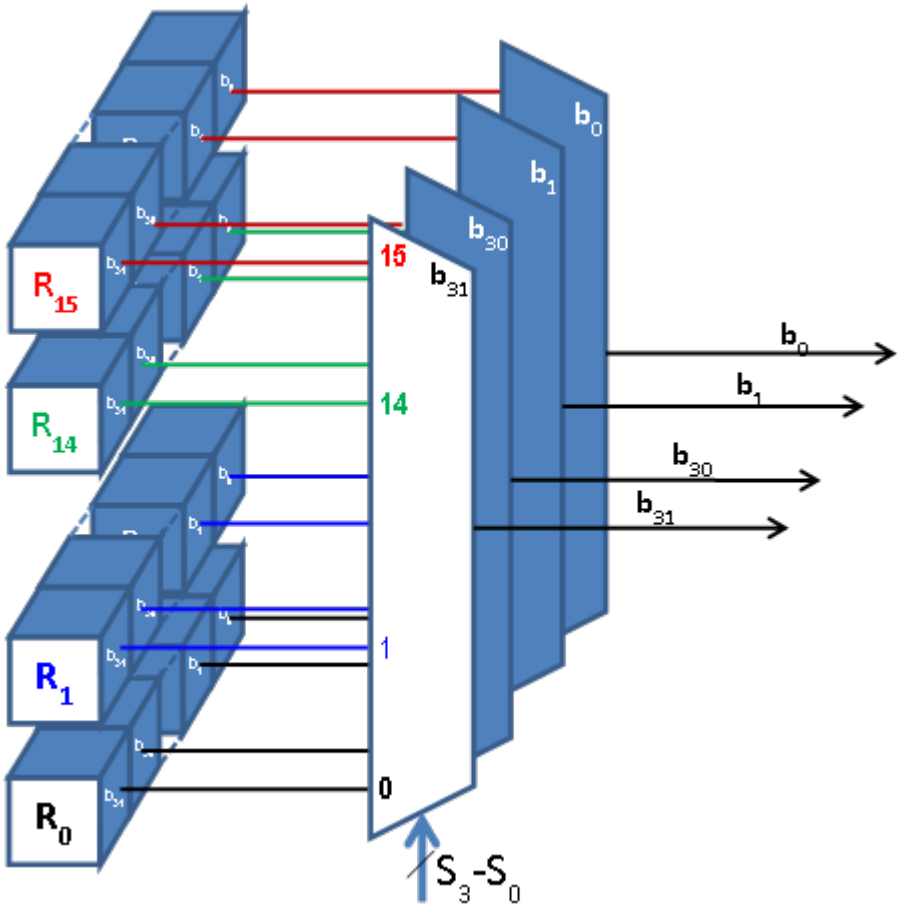
e.g., n=4 k=2:

S1	S0	Register selected
0	0	A
0	1	B
1	0	C
1	1	D



A bus system for four registers using multiplexers

Common Bus System for 16 registers of 32 bits each



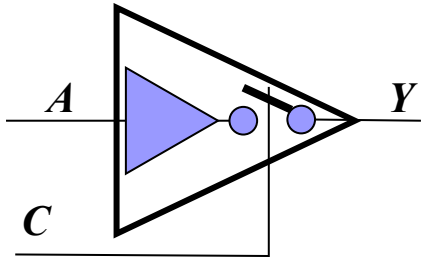
Three-State Buffers

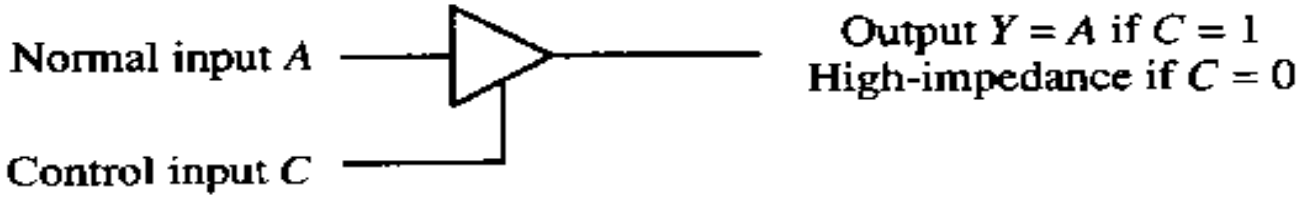
- A bus system may also be constructed with three-state gates instead of multiplexers.
- A three-state gate is a digital circuit that exhibits three states:
  - Two of these states correspond to signals equivalent to logic 1 and 0, as in a conventional gate.
  - The third state is high-impedance state.
- The **high-impedance** state makes the gate behaves like an open circuit (think of this state as if the output of the gate is “disconnected”).

Three-State Buffers (cont.)

- Three-state gates can come in several forms, such as AND gates, NAND gates, etc.
- However, there is one that is most commonly used in the design of bus systems: the **three-state buffer** gate.

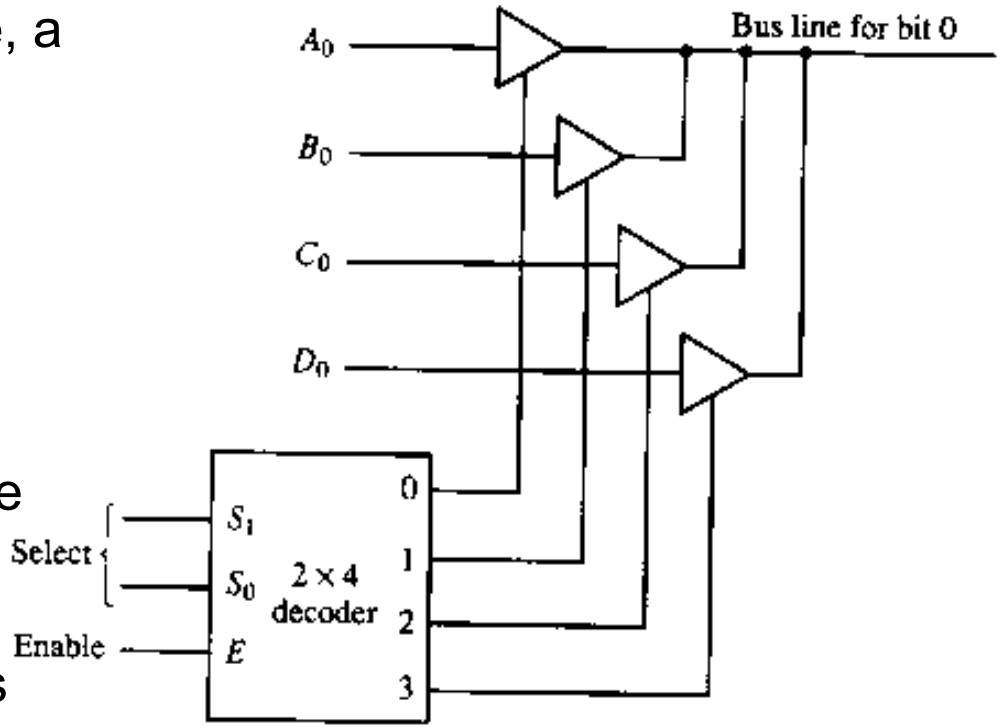
Control Input (C)	Data Input (A)	Output (Y)
1	0	**0**
1	1	**1**
0	**x**	**High-impedance**





A bus line with three-state buffers

- To ensure that no more than one control input is active at any given time, a decoder is used.
- When the *Enable* input of the decoder is 0, all of its four outputs are reset to 0 ⇒ The bus line is in a high-impedance state (logically "disconnected").
- When the *Enable* input is 1, the input of the selected buffer is transferred to the bus line. In this case, the above circuit acts like a 4 × 1 multiplexer.



- To construct a common bus system for four registers of *n* bits each using three-state buffers, *n* circuits like the one in figure are needed. However, only one decoder is required as common decoder for all of them.

Memory Transfer

- In the following, we will symbolize a memory word by the letter *M* .
- The *address register* holding the memory address of the word to be accessed will be symbolized by *AR*.
- The *data register* to hold word to be read from or written to the memory unit will be symbolized by *DR* .
- A memory **read** operation is stated as follows:

  Read: $DR \leftarrow M[AR]$, to mean

  if *the control function (memory control signal)* **Read** *is 1,* **then** *transfer the contents of the word* ***from*** *the memory location specified by address stored in register AR* ***into*** *register DR .*
- A memory **write** operation is stated as follows:

  Write: $M[AR] \leftarrow DR$, to mean

  if *the control function (memory control signal)* ***Write*** *is 1,* **then** *transfer the word stored in register DR to the memory word located at the address which is stored in register AR .*

Arithmetic Micro-operations

The micro-operations most often encountered in digital computers are classified into four categories:

1. Register **transfer** micro-operations: they transfer binary data from one register to another.
2. **Arithmetic** micro-operations: they perform arithmetic micro-operations, such as additions and subtractions for instance, on digital data stored in registers.
3. **Logic** micro-operations: they perform bit manipulation operations on data stored in registers.
4. **Shift** micro-operations: they perform shift operations on data stored in registers.

- We have just seen register **transfer** micro-operations.
- In what follows, we will introduce each of the remaining three types of micro-operations.

Add Micro-operation

- ... The micro-operation defined by the statement:

$$R3 \leftarrow R1 + R2$$

denotes an **add** micro-operation.

- It states that the content of register *R*1 is added to the content of register *R*2 and the sum is transferred to register *R*3.

Subtract Micro-operation

- Subtracting the content of register *R*2 from the content of register *R*1 and storing the result in register *R*3 can be symbolized by the following two possible notations:

$$R3 \leftarrow R1 - R2 \qquad \text{or} \qquad R3 \leftarrow R1 + \overline{R2} + 1$$

- $R2$ denotes the complement of the data stored in register $R2$.
- **Next** Table lists the most common basic micro-operations

Common Arithmetic Micro-operations

#	Symbolic Designation (RTL)	Description
1.	$R3 \leftarrow R1 + R2$	Content of R1 plus R2 transferred to R3
2.	$R3 \leftarrow R1 - R2$	Content of R1 minus R2 transferred to R3
3.	$R2 \leftarrow \overline{R2}$	Complement the content of R2 (1's complement)
4.	$R2 \leftarrow \overline{R2} + 1$	2's complement the content of R2 (negate)
5.	$R3 \leftarrow R1 + \overline{R2} + 1$	R1 plus the 2's complement of R2 (subtraction)
6.	$R1 \leftarrow R1 + 1$	Increment the content of R1 by 1
7.	$R1 \leftarrow R1 - 1$	Decrement the content of R1 by 1

Binary Adder

- To implement the add micro-operation in hardware, we need at least two registers to hold the two numbers to be added, possibly a third register to store the result of the operation, and a combinational circuit to perform the arithmetic addition.
- A combinational circuit that adds two bits and a carry bit is called a full-adder (FA) {refer to Chapter 1}.
- The combinational circuit that performs the arithmetic sum of two n-bit numbers is called an *n*-bit binary adder.
- Typically, an *n*-bit binary adder is constructed with *n* full-adder circuits connected in cascade, with the output carry from one full-adder connected to the input carry of the next full-adder.

ADDING BINARY NUMBERS

Adding two bits:

0+	0+	1+	1+
0	1	0	1
0	1	1	10

The binary number **10** is equivalent to the decimal **2**

Carry (over) **Sum**

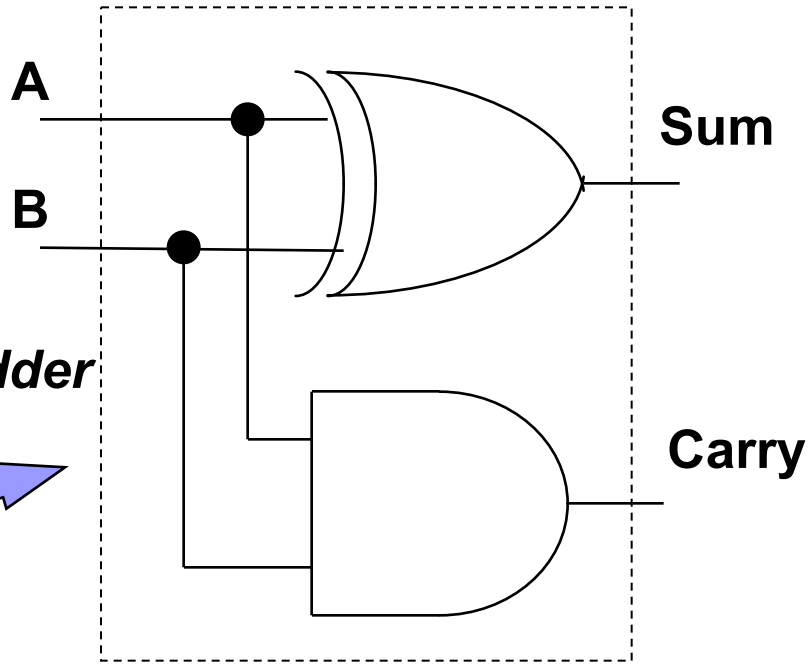
Truth table

Inputs A	Inputs B	Outputs Carry	Outputs Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$\text{Sum} = A \oplus B$$

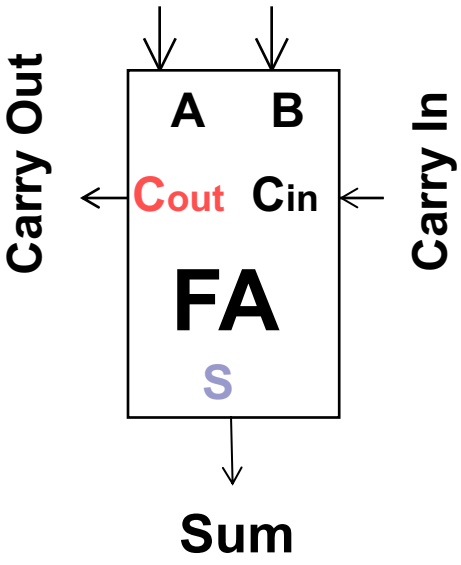
$$\text{Carry} = A \cdot B$$

Half-Adder circuit

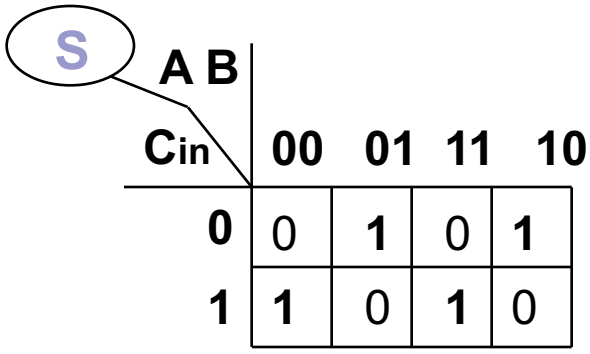


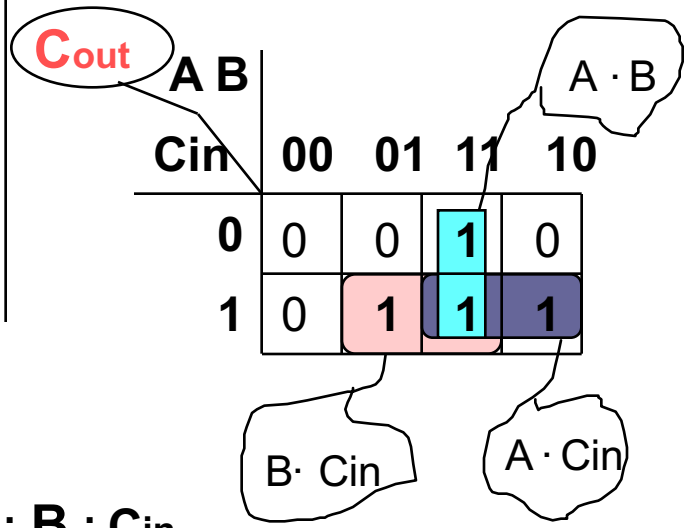
Full Adder

Bits of the same rank of the two numbers



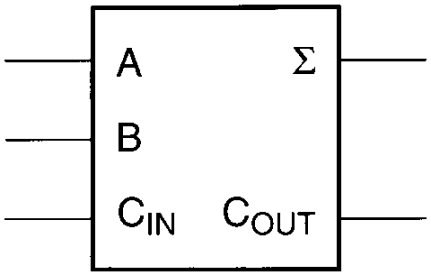
Inputs			Outputs	
A	B	Cin	Cout	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1





$$S = \overline{A} \cdot \overline{B} \cdot C_{in} + \overline{A} \cdot B \cdot \overline{C_{in}} + A \cdot \overline{B} \cdot \overline{C_{in}} + A \cdot B \cdot C_{in}$$

$$C_{out} = A \cdot B + B \cdot C_{in} + A \cdot C_{in}$$



Full Adder (FA)

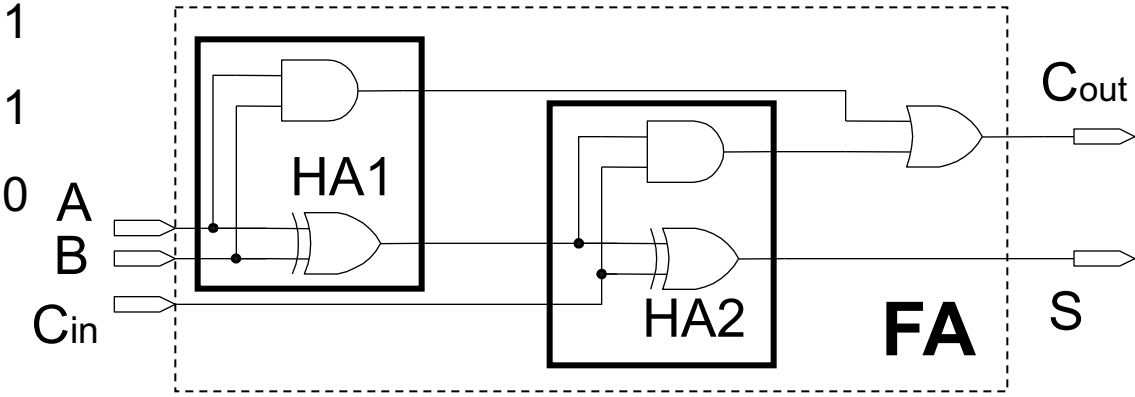
employing Half Adders (HA)

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	0

•Equations are modified as follows:

$$C_{out} = ((A \oplus B) \cdot C_{in}) + (A \cdot B)$$

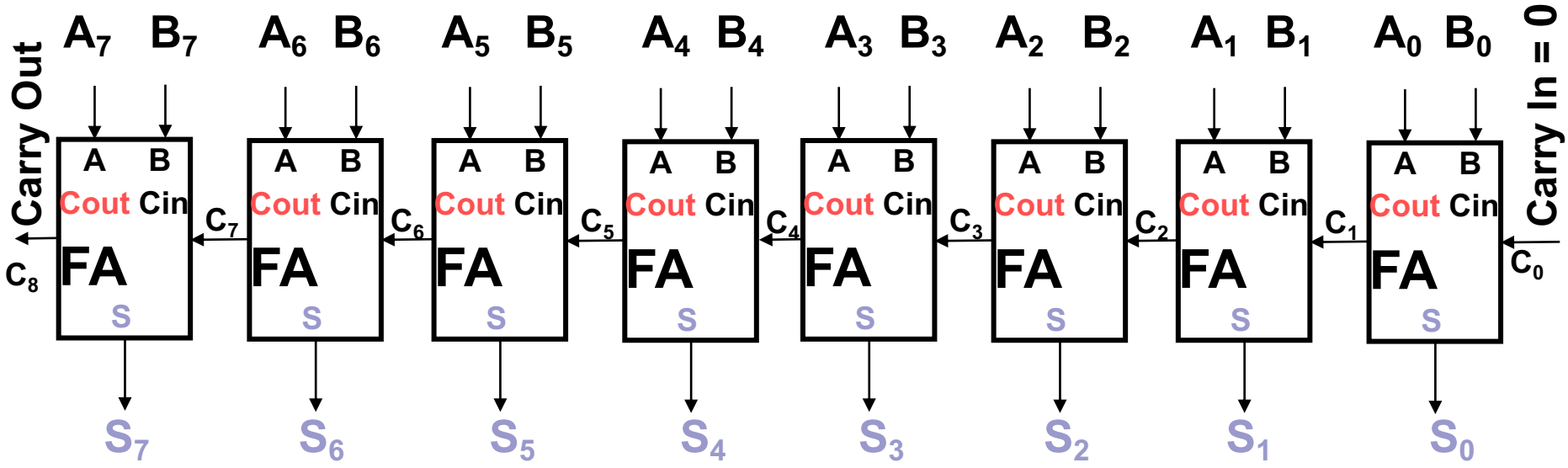
$$S = A \oplus B \oplus C_{in}$$



Adding multi-bit numbers:

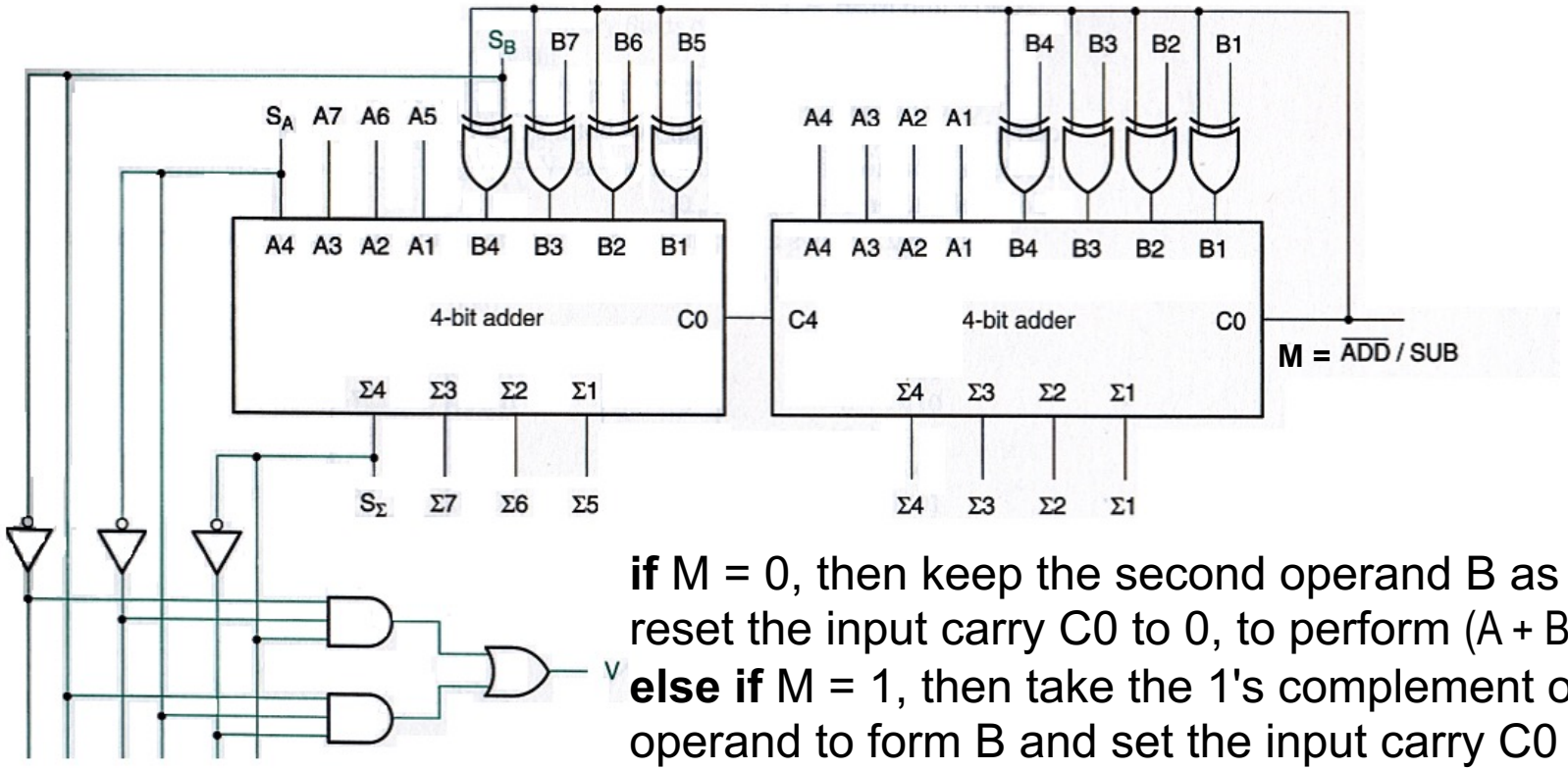
108_{10} + → 0 1 1 0 1 1 0 0 +
90_{10} → 0 1 0 1 1 0 1 0
198_{10} ← 1 1 0 0 0 1 1 0 ⇦ Sum
0 1 1 1 1 0 0 0 ⇦ Carry

$A_7\ A_6\ A_5\ A_4\ A_3\ A_2\ A_1\ A_0$ +
$B_7\ B_6\ B_5\ B_4\ B_3\ B_2\ B_1\ B_0$
$S_7\ S_6\ S_5\ S_4\ S_3\ S_2\ S_1\ S_0$

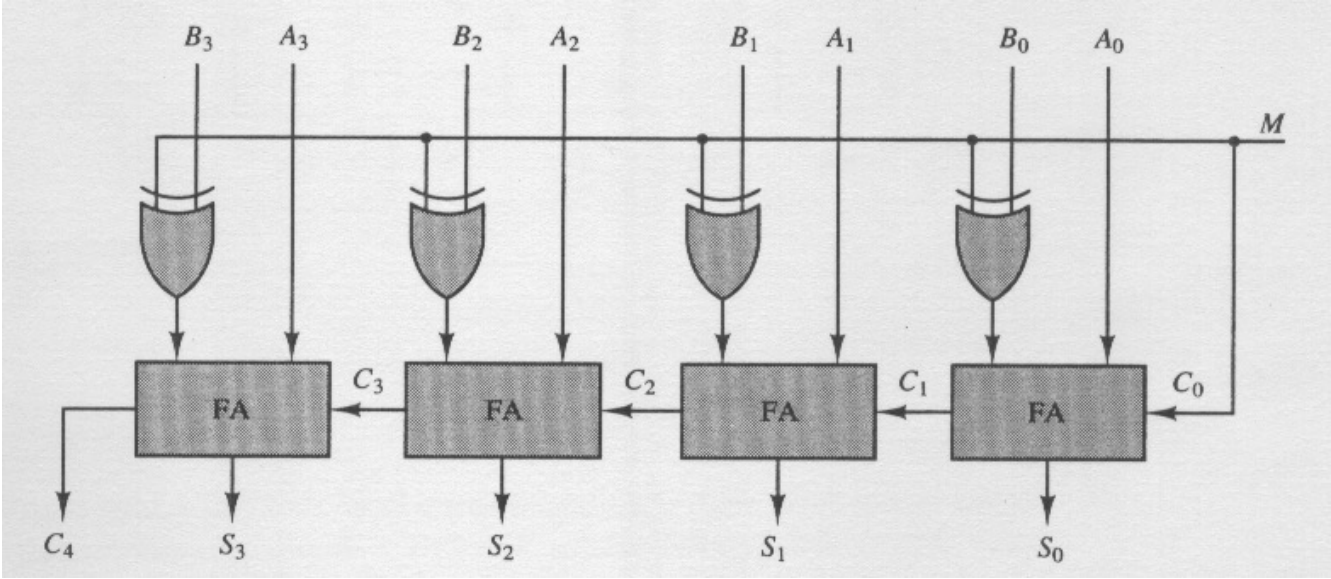


8 – bit Binary Adder / Subtractor with Overflow Detector

The operation (A-B) is accomplished by performing the operation (A + $\overline{B}$ + 1), which is in essence an addition operation => the addition and subtraction operations can be combined into one common circuit that uses an input control bit, say M, to indicate the type of the desired operation.



if M = 0, then keep the second operand B as it is and reset the input carry C0 to 0, to perform (A + B + 0) = (A + B).
else if M = 1, then take the 1's complement of the second operand to form B and set the input carry C0 to 1, to perform (A + $\overline{B}$ + 1) = (A-B).
The above two statements can be combined into one as follows: perform A + (M $\oplus$ B) + M



- For unsigned numbers,
  - When $M = 0$ (addition micro-operation), $S_{n-1} \ldots S_1 S_0$ gives the result of $(A + B)$. An overflow occurs when $C_n = 1$.
  - When $M = 1$ (subtraction micro-operation) and if $A \geq B$, then $S_{n-1} \ldots S_1 S_0$ gives the result of $(A - B)$. An overflow cannot possibly exist in this case.
  - When $M = 1$ (subtraction micro-operation) and if $A < B$, then $S_{n-1} \ldots S_1 S_0$ gives the result of the 2's complement of $(B - A)$. An overflow cannot possibly exist in this case.
- For signed numbers,
  - When $M = 0$ (addition micro-operation), $S_{n-1} \ldots S_1 S_0$ gives the result of $(A + B)$. An overflow occurs when $C_n \oplus C_{n-1} = 1$.
  - When $M = 1$ (subtraction micro-operation), $S_{n-1} \ldots S_1 S_0$ gives the result of $(A - B)$. An overflow occurs when $C_n \oplus C_{n-1} = 1$.